

实验三 网络编程

一、实验目标

- 练习使用编程语言提供的网络接口；
- 自行设计方案解决 TCP 的“粘包”问题，复习 TCP 面向字节流的特点；
- 编写简单的客户-服务端程序、web 服务器；
- 编写基于 UDP 的简单 ping 程序。

二、提前阅读

2.1 实验报告书写要求（重要！）

- 1、需要回答问题的部分会使用**红色**标出，例如“**请使用文字与截图（可选）回答**”，没有红色标注的部分不需要回答；
- 2、请将你的回答中的文字设置为“**深蓝色**”，方便批改；
- 3、回答问题时不抄袭、不直接复制大模型的回答、不堆砌文字、保持简洁、体现自己的思考。

2.2 关于编程语言

我们的 demo 示例使用了 Python 语言。除了 Python 语言，感兴趣的同学可以使用 C、C++、Java、go 等编程语言，我们不做编程语言的限制。对于使用其他语言的同学，可以改掉提交实例的后缀，比如我们要求提交 test.py 文件，而你使用 Java 语言，则提交 test.java 文件，其他语言依此类推。

2.3 其他注意事项

- 1、可以不参考 demo 代码的注释和示例，可以自由发挥，但是必须符合题目要求。

2、硬编码（按题目要求直接把结果一个个 print 出来，然后截图）的，本周的作业全部计零分。

3、思考题中要求临时修改代码或运行观察的内容，仅用于临时的实验现象验证；最终提交的代码应恢复为题目要求，报告中用简短文字说明观察结果即可。

三、实验要求：

1、简单 UDP 编程

利用 UDP 协议，完成下列的程序要求，客户端代码放在 code/Q1_UDPClient.py，服务端代码放在 code/Q1_UDPServer.py

注：我们提供 code/Q1_UDPClient.py 作为示例，该代码已经写好，无需修改。

程序要求：

- 1) 服务端开启 UDP 监听；
- 2) 客户从其键盘读取一行字符（小写字母，小于 1024 字节）并将该数据向服务端发送；
- 3) 服务端接受该数据（接受缓冲为 1024 字节）并将这些字符转换为大写；
- 4) 服务端将修改的数据发送给客户；
- 5) 客户端接受修改的数据并在其控制台上将该行显示出来。

注：养成良好编程的习惯，记得关闭 IO 流！

思考题：请使用文字与截图（可选）回答

1) 在“简单 UDP 编程”中，客户端发送一次数据后，服务端一定会收到吗？为什么？（分“现实应用”、“本机测试环境”两个角度考虑。本机测试环境通常比较稳定，现实应用中情况如何呢？）

本机测试：本机环境比较稳定，基本一定会收到

现实应用：显示网络环境含有丢包，网络拥塞，路由变化，设备故障，不一定会收到

(((((((((((亲爱的老师/助教，平常都是用的透明终端，后知后觉可能会造成一些批阅的麻烦，实在抱歉))))))))))))))

```
code 4 PORT = 12000
5 BUFFER_SIZE = 1024
pycache_ Q1_UDPClient.py Q2_TCPServer.py Q3_UDPClient.py Q4_TCPClient.py Q4_TCPServer.py
Q1_UDPClient.py Q2_TCPClient.py Q3_TCPClient.py Q4_TCPClient_modi.py Q4_TCPServer_modi.py
> py Q1_UDPClient.py
UDP server listening on 127.0.0.1:12000
Q3_TCPClient.py
Q3_TCPServer.py
Q4_TCPClient_modi.py
Q4_TCPClient.py
Q4_TCPServer_modi.py
Q4_TCPServer.py
udp_pinger_code
> __pycache__
UDP_pinger_client.py
UDP_pinger_server.py
web_server_code
> __pycache__
404.html
cd /code
cd /code
simple_web_server.py
web_server_sample.py
E208-实验3-实验要求-网络编程.docx
SSE208-实验3-实验要求-网络编程.docx
py Q1_UDPClient.py
Input lowercase sentence:This is a string sent to server
THIS IS A STRING SENT TO SERVER
~/obnotes/25_2/cn/课件/实验3-网络编程 code main !185 755 >
```

2) 请临时修改代码并测试：把服务端接收缓冲区从 1024 字节临时改成 5 字节，然后让客户端发送超过 5 字节的字符串。观察客户端收到的结果，并说明原因。完成观察后，请将代码恢复为实验要求中的 1024 字节。

```
19
20 if __name__ == "__main__":
21     main()
22
web_server_code
> __pycache__
404.html
cd /code
cd /code
simple_web_server.py
web_server_sample.py
E208-实验3-实验要求-网络编程.docx
SSE208-实验3-实验要求-网络编程.docx
py Q1_UDPClient.py
Input lowercase sentence:This is a string sent to server
THIS IS A STRING SENT TO SERVER
py Q1_UDPClient.py
Input lowercase sentence:This is a string
THIS
py Q1_UDPClient.py
Input lowercase sentence:abcdefg
ABCDEF
~/obnotes/25_2/cn/课件/实验3-网络编程 code main !185 755 >
```

服务端只收到了 5 字节

这是因为 recvfrom(5)的话一次只能接受 5 字节,剩下的被截断了。

2、简单 TCP 编程

利用 TCP 协议，完成下列的程序要求，客户端代码放在 code/Q2_TCPClient.py，服务端代码放在 code/Q2_TCPServer.py

程序要求：

1) 服务端开启 TCP 监听；

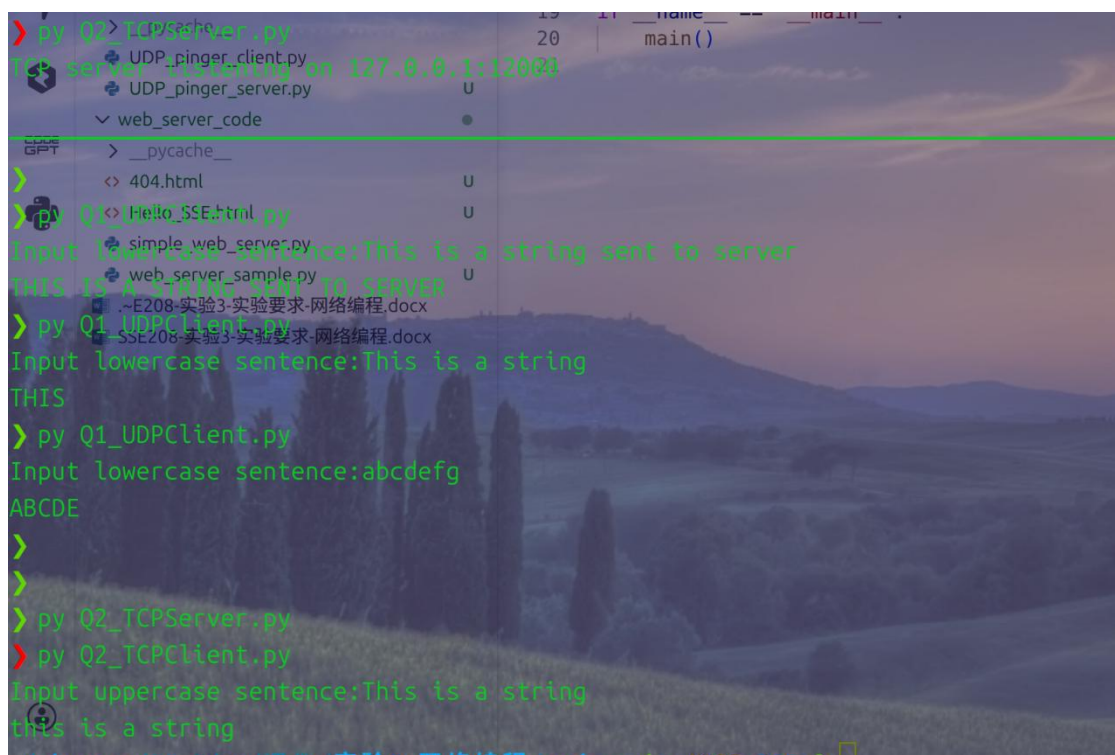
- 2) 客户从其键盘读取一行字符 (大写字母, 小于 1024 字节) 并将该数据向服务端发送;
- 3) 服务端接受该数据 (接受缓存为 1024 字节) 并将这些字符转换为小写;
- 4) 服务端将修改的数据发送给客户, 并关闭连接;
- 5) 客户端接受修改的数据并在其控制台上将该行显示出来, 并关闭连接;
- 6) 服务端继续等待下次连接;

注: 养成良好编程的习惯, 记得关闭 IO 流!

思考题: 请使用文字与截图 (可选) 回答

1) Q2 中, 服务端为什么需要先创建监听 socket, 再通过 accept 得到一个新的连接 socket? 这两个 socket 分别负责什么?

先创建监听 socket 因为服务器必须先绑定特定 ip 和端口进入监听状态, 才能等待客户端连接. 这个服务器通过 accept() 收到的 socket 是专门负责与客户端进行数据收发, 而创建的监听 socket 负责连接.



```
> py Q2_TCPServer.py
TCP server is listening on 127.0.0.1:12000
> py Q1_UDPClient.py
Input lowercase sentence: This is a string sent to server
THIS IS A STRING SENT TO SERVER
> py Q1_UDPClient.py
Input lowercase sentence: This is a string
THIS
> py Q1_UDPClient.py
Input lowercase sentence: abcdefg
ABCDE
>
>
> py Q2_TCPServer.py
> py Q2_TCPClient.py
Input uppercase sentence: This is a string
this is a string
```

2) 请实际修改并测试: 在 Q2 客户端第一次收到服务端响应后, 不关闭 socket, 尝试继续发送第二条消息。观察程序表现, 并解释原因。完成观察后,

请恢复代码，使其符合原始程序要求。

```
7
8 def main():
9     """实现一个 TCP 客户端：发送一行大写字母，并接收服务端返回的小写结果。"""
10
11     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
12
13         client_socket.connect((SERVER_HOST, SERVER_PORT))
14
15         message = input("Input uppercase sentence:")
16         client_socket.sendall(message.encode())
17         modified_message = client_socket.recv(BUFFER_SIZE)
18         print("first receive:",modified_message.decode())
19
20         message = input("Input uppercase sentence:")
21         client_socket.sendall(message.encode())
22         modified_message = client_socket.recv(BUFFER_SIZE)
23         print("second receive:",modified_message.decode())
24
25
26 if __name__ == "__main__":
27     main()
28
```

```
> py Q2_TCPClient.py
17 modified_message = client_socket.recv(BUFFER_SIZE)
18 print("first receive:",modified_message.decode())
19
20 message = input("Input uppercase sentence:")
21 client_socket.sendall(message.encode())
22 modified_message = client_socket.recv(BUFFER_SIZE)
23 print("second receive:",modified_message.decode())
24
25
26 if __name__ == "__main__":
27     main()
28
KeyboardInterrupt
> py Q2_TCPClient.py
Input uppercase sentence:First String
First receive: First string
Input uppercase sentence:Second String
second receive:
~/obsidian/25_2/cn/课件/实验3-网络编程/code/Q2_TCPClient.py, line 10, in main
```

第二次发送的消息没有受到服务端发送回来的内容。

服务器在 while True 下面,每一次受到客户端数据之后,会调用 handle_client(),但是 handle_client()内部只会调用一次 recv(),这意味着每次进入 handle_client()之后处理一条信息之后,会把这条连接关闭,对于第二次接受的内容,已经没有了之前的连接,当然无法处理,

3) 从客户端程序的角度看, send 或 sendall 返回说明数据已经到达了哪一层? 为什么这不能证明服务端应用程序已经接收并处理了这段数据?

说明已经到达了本机的 socket 发送缓冲区,也就是 tcp 协议栈.并不能证明已经被服务端应用程序接受,后面还要经过网络传输->服务端接受缓冲区和服务端调用 recv()接受然后才能被处理。

3、TCP 不中断续传

在 2) 中，服务端处理完一次响应后就关闭了本次连接，等待下次连接。在本小节中，我们要求服务端可以不断地处理客户端数据，直到客户端要求断开连接。

请利用 TCP 协议，实现下面的程序要求，客户端代码放在 `code/Q3_TCPClient.py`，服务端代码放在 `code/Q3_TCPServer.py`

程序要求：

- 1) 服务端开启 TCP 监听；
- 2) 客户从其键盘读取一行字符（**大写字母，小于 1024 字节**）并将该数据向服务端发送；
- 3) 服务端接受该数据（**接受缓存为 1024 字节**）并将这些字符转换为小写；
- 4) 服务端将修改的数据发送给客户；
- 5) 客户端持续步骤 2)，不断地发送数据给服务端，输入“bye”结束发送，并关闭连接；

```
KeyboardInterrupt
> py Q3_TCPClient.py
input the sentence to be transformed:A
1
input the sentence to be transformed:BA8
bab
input the sentence to be transformed:bye
~/notes/obsidian_notes/25_2/cn/课件/实验3-网络编程/code main !185 ?55 >

Traceback (most recent call last):
  File "/home/loviya/notes/obsidian_notes/25_2/cn/课件/实验3-网络编程/code/Q3_TCPClient.py", line 27, in <module>
    main()
  File "/home/loviya/notes/obsidian_notes/25_2/cn/课件/实验3-网络编程/code/Q3_TCPClient.py", line 14, in main
    message=input("input the sentence to be transformed:")
KeyboardInterrupt
```

```
KeyboardInterrupt
> py Q3_TCPClient.py
input the sentence to be transformed:A
1
input the sentence to be transformed:BA8
bab
input the sentence to be transformed:bye
~/notes/obsidian_notes/25_2/cn/课件/实验3-网络编程/code main !185 ?55 >

Traceback (most recent call last):
  File "/home/loviya/notes/obsidian_notes/25_2/cn/课件/实验3-网络编程/code/Q3_TCPClient.py", line 27, in <module>
    main()
  File "/home/loviya/notes/obsidian_notes/25_2/cn/课件/实验3-网络编程/code/Q3_TCPClient.py", line 14, in main
    message=input("input the sentence to be transformed:")
KeyboardInterrupt
```

6) 服务端不断处理客户端发来的数据,当收到“bye”的时候关闭与客户端的连接,继续等待下一个 TCP 连接;

注: 养成良好编程的习惯,记得关闭 I/O 流!

思考题: 请使用文字与截图(可选)回答

1) 请实际运行并观察两种情况：第一，客户端输入 bye 正常退出；第二，客户端程序被直接关闭或中断。服务端分别可能看到什么？你的服务端代码应该如何处理？

```
# 3. 如果 recv 返回空字节, 说明客户端已经关闭连接。
if not data:
    print(f"Client closed connection:")
    break
# 2. 如果客户端发送 QUIT MESSAGE, 结束本次连接。
message=data.decode()
if message==QUIT_MESSAGE:
    print(f"Client sent quit message,closing this connection")
    break
```

根据我的设置,bye 退出的时候服务收到了”bye”,中断连接并且显示了” Client sent quit message,closing this connection”而 ctrl+c 中断客户端,recv()返回了空字节,退出连接后显示了” Client closed connection:”;服务器都是退出当前连接,监听下次连接

2) 在 Q3 中,如果客户端连续发送多条消息,服务端能否把“一次 recv”直接等同于“客户端输入的一行”?为什么?临时修改代码,测试会发生什么,并简单记录,完成观察后,请恢复代码,使其符合原始程序要求(提示:可以临时修改客户端,在同一连接中连续发送两条消息且不等待服务端响应,观察服务端一次 recv 得到的内容。该问题实际上就是下面“4、TCP 粘包问题”。)

```
3         #         break
4         # # 5. 退出前关闭连接。
5         #         print(modified_message.decode())
6         client_socket.sendall("StringONE".encode())
7         client_socket.sendall("StringTWO".encode())
8
9         modified_message=client_socket.recv(BUFFER_SIZE)
10        print("From server:",modified_message.decode())
11    if __name__ == "__main__":
12        main()
13
```

```
> py Q3_TCPClient.py
From server: stringonestringtwo
```

不能,因为 TCP 是字节流协议,只保证数据按顺序到达,但不保留每条消息的边界。

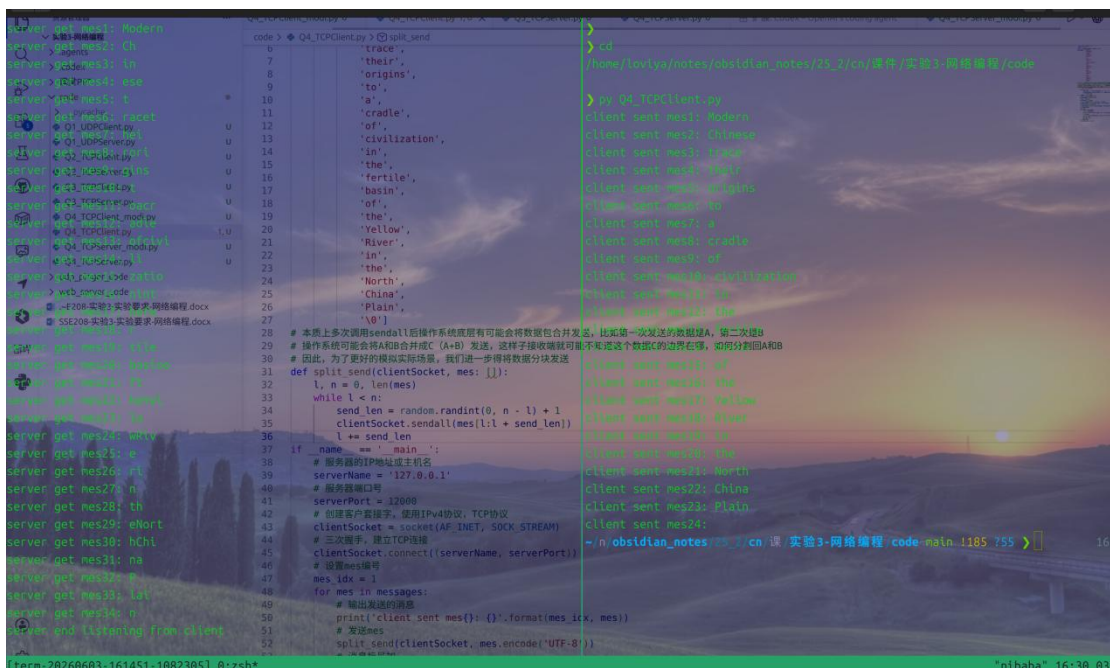

```
client sent mes1: Modern
The server is ready to receive
server get mes1: Mod
server get mes2: e
server get mes3: r
server get mes4: n
server get mes5: Chin
server get mes6: ese
server get mes7: tra
server get mes8: c
server get mes9: e
server get mes10: the
server get mes11: i
server get mes12: r
server get mes13: origin
```

图 1：客户端运行结果

图 2：服务端运行结果（以上为示例，请以实际运行结果为准）

请问为什么客户端发送给服务端时，服务端显示的 mes 与客户端不同？请使用文字与截图（可选）回答。

因为这两个 mes 的含义不同,客户端会遍历 messages 整个数组,对于里面每个 str 可能分成多个小块发送;而服务端 recv() 也是每次接收随机字节数量的缓冲区,这就导致客户端每次发送的小块可能一次被接受,也可能多次被接受..此外,客户端的 mes 是遍历 messages 的 str 的序号,服务端的 mes 是收到的 str 的序号.本质原因还是 tcp 是字节流协议.



请你在原来的代码（code/Q4_TCPServer.py 和 code/Q4_TCPClient.py）的基础上，解决这种问题，方法不限，硬编码除外，并保存为code/Q4_TCPServer_modi.py 和 code/Q4_TCPClient_modi.py。

程序要求：

客户端发送的每条应用层消息应当能够被服务端按原顺序还原并打印；消息边界处理方法不限（如分隔符、长度前缀等），但不能依赖一次 recv 恰好对应一条消息，如下图所示：

```

client sent mes1: Modern
client sent mes2: Chinese
client sent mes3: trace
client sent mes4: their
client sent mes5: origins
client sent mes6: to
client sent mes7: a
client sent mes8: cradle
client sent mes9: of
client sent mes10: civilization
client sent mes11: in
client sent mes12: the
client sent mes13: fertile

```

图 3：客户端发送示例

```
The server is ready to receive
server end listening from client
server get mes1: Modern
server get mes2: Chinese
server get mes3: trace
server get mes4: their
server get mes5: origins
server get mes6: to
server get mes7: a
server get mes8: cradle
server get mes9: of
server get mes10: civilization
server get mes11: in
server get mes12: the
server get mes13: fertile
```

图 4：服务端接受示例，应当与客户端保持一致

思考题：请使用文字与截图（可选）回答

1) 你在 Q4_TCPClient_modi.py 和 Q4_TCPServer_modi.py 中采用了什么方法划分消息边界？请说明它如何从连续字节流中恢复出一条完整消息。

客户端遍历 messages 数组时,对于每个 str 会先编码成 utf-8 然后发送一个四字节数字作为这个 str 的长度到客户端,然后服务端在接收消息的时候,会先解析出这个 header(长度),然后再调用 `recv_exact(connectionSocket,mes_len)`来读取后面具体长度的内容并 decode 回 utf-8.



```
def recv_exact(connectionSocket, size):
    """
    从TCP字节流中准确读取size个字节。
    TCP不保留消息边界,所以不能假设一次recv就能收到完整数据。
    """
    data = b''
    while len(data) < size:
        packet = connectionSocket.recv(min(get_random_buffer_size(), size - len(data)))
        # 客户端关闭连接时,recv会返回b''
        if not packet:
            return None
        data += packet
    return data
```



```

def send_one_message(clientSocket,mes:str):
    """
    发送一条完整应用层消息。
    方法:消息体前加4字节长度前缀。
    """
    mes_data=mes.encode('utf-8')
    # !I 表示网络字节序的4字节无符号整数
    header=struct.pack('!I',len(mes_data))
    split_send(clientSocket,header+mes_data)
if __name__ == '__main__':
    # 服务器的IP地址或主机名
    serverName = '127.0.0.1'
    # 服务器端口号
    serverPort = 12000
    # 创建客户套接字,使用IPv4协议,TCP协议
    clientSocket = socket(AF_INET, SOCK_STREAM)
    # 三次握手,建立TCP连接
    clientSocket.connect((serverName, serverPort))
    # 设置mes编号
    mes_idx = 1
    for mes in messages:
        # 输出发送的消息
        print('client sent mes{:}: {}'.format(mes_idx, mes.replace('\0','')))
        # 发送mes
        # split_send(clientSocket, mes.encode('UTF-8'))
        send_one_message(clientSocket,mes)
        # 消息标号加一
        mes_idx += 1
    # 关闭socket
    clientSocket.close()

```

2)如果服务端一次 recv 收到“两条半消息”，或者一次只收到“半条消息”，你的程序应该分别如何处理？

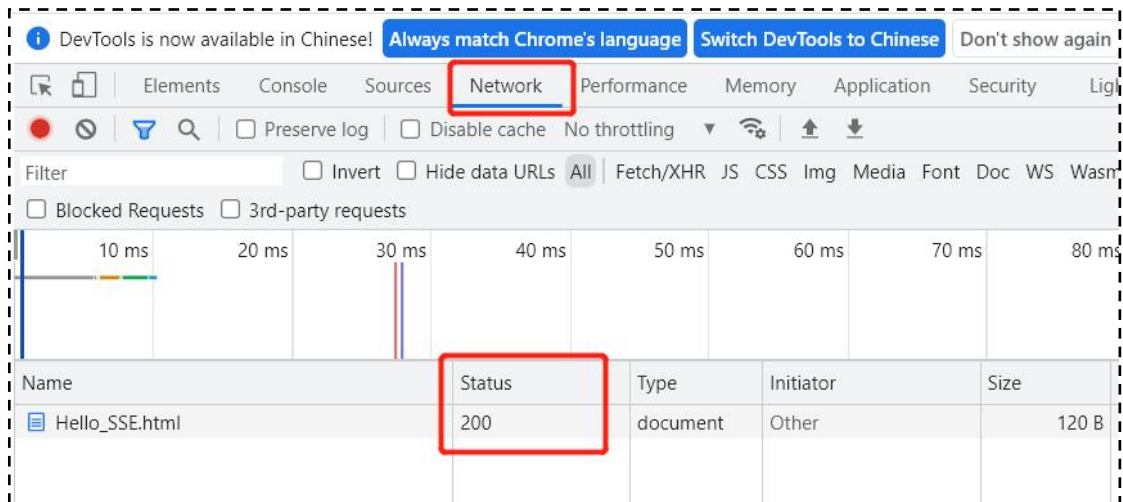
如果服务端一次 recv() 只收到“半条消息”，程序不会马上打印，而是继续调用 recv()，把后续收到的数据继续拼接起来，直到累计 收到的字节数等于长度字段中记录的消息长度，才会认为这一条消息接收完整，然后再解码和打 印。

如果服务端一次 recv() 收到“两条半消息”，在本程序中由于 recv_exact() 每次只读取当前需要的字节数，所以第一条消息读满后就会先打印第一条，剩余的数据会留在 TCP 接收缓冲区中，下一轮再继续读取下一条消息的长度字段和消息体。因此不会把两条消息混在一起打印。

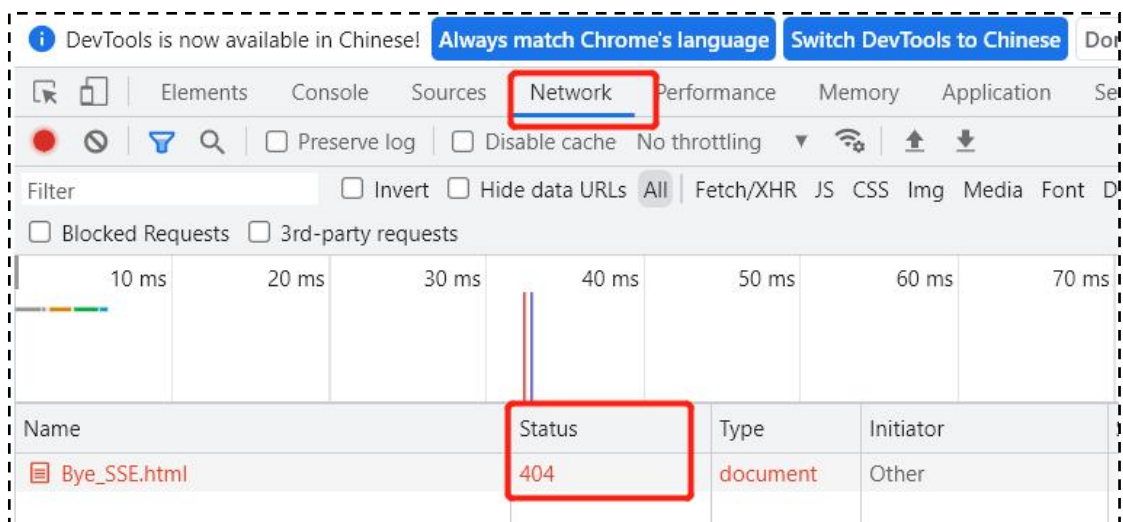
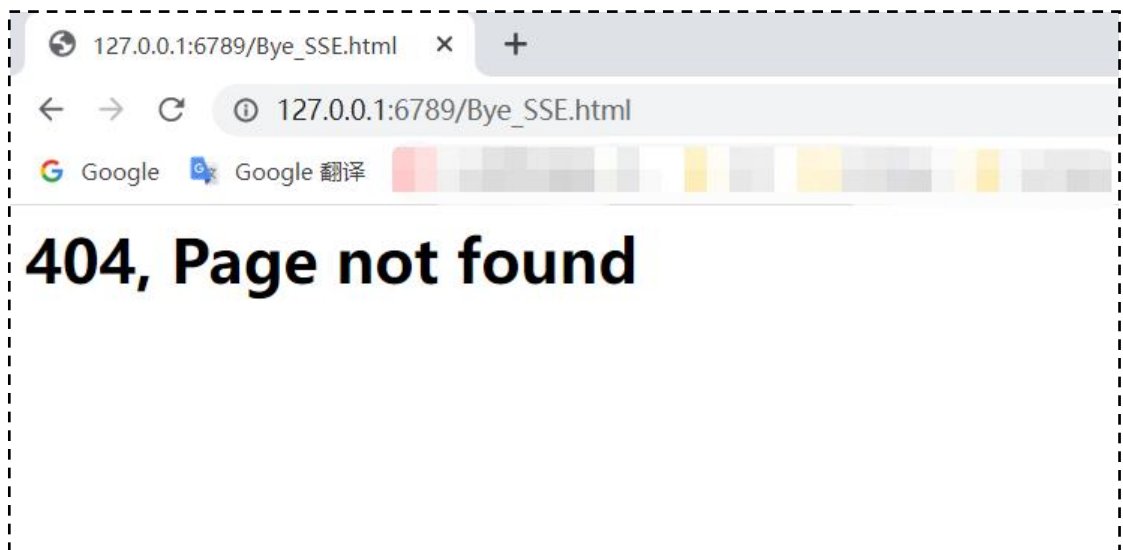
5、简单 Web 服务器

要求：

在前面的实验中，你已经学会 TCP 编程。在本实验



2、输入服务器不存在的资源：比如“http://127.0.0.1:6789/Bye_SSE.html”，页面应该显示“404.html”文件的内容，状态码为 404。

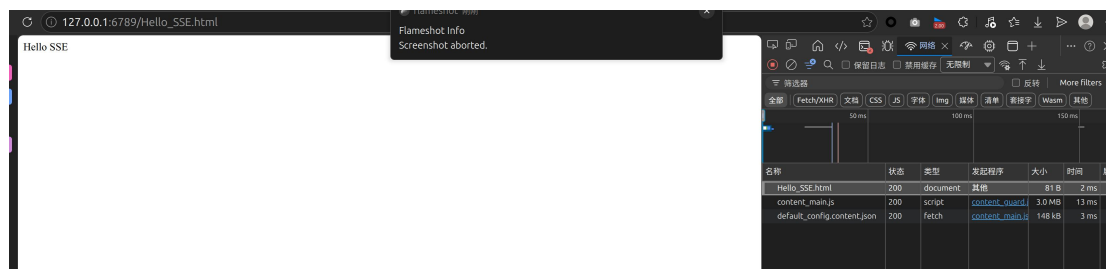


提交内容：

1、服务器代码，保存为“web_server_code/simple_web_server.py”

ok

2、客户端浏览器输入“http://127.0.0.1:6789/Hello_SSE.html”后的网页和状态码（200）截图，应与上述例子相似，将截图放在此处。



3、客户端浏览器输入服务器不存在的资源，比如“http://127.0.0.1:6789/Bye_SSE.html”后的网页和状态码（404）截图，应与上述例子相似，将截图放在此处。



注：“Hello_SSE.html”是你放在服务器目录中的文件。在使用浏览器访问你的服务时，你需要使用服务器代码中绑定的端口号。比如，在上面的例子中，我们服务器绑定端口号 6789，浏览器应该显示 Hello_SSE.html 的内容。如果省略“:6789”，浏览器将使用默认端口 80，除非你的服务器就是监听 80 端口，才会从服务器获取网页，但我们不建议使用常用的端口。

思考题：请使用文字与截图（可选）回答

1) 一个最小 HTTP 响应通常由状态行、响应头、空行和响应体组成。它们分别有什么作用？为什么空行不能省略？

状态行：说明服务器对客户端请求的处理结果

响应头：包含本次响应的一些信息，比如内容类型，长度和连接类型

空行：分隔响应头和状态行，便于区分

响应体：真正返回给客户端的信息

空行是 http 协议规定的，响应头后面必须有个空行，要不然可能无法分清响应头和状态行

2) 你的代码如何从 HTTP 请求中解析文件名？文件存在和不存在时，响应构造有什么区别？

首先通过 `message.splitlines` 获取第一行 `GET /Hello._SSe.html HTTP/1.1`, 然后通过 `split()[1]` 获取 `/hello_SSE.html` 通过 `[1:]` 分片获得文件名。

我的代码会测试能否打开这个文件，如果打开正常返回资源，不能打开就默认打开 `404.html` 并返回

6、UDP ping 程序

概述：

我们知道 ping 程序位于网络层，基于 ICMP 协议，其允许客户端机器发送一个数据包到远程机器，并使远程机器响应回客户（称为回显）。另外，ping 程序还允许主机计算它到其他机器的往返时间。在上次实验中，你已经学会如何使用 UDP 编程。考虑到 ICMP 的复杂性，本次实验，你将基于 UDP 简单复现一个 ping 客户端应用程序。

ping 服务器程序代码说明：

“`udp_pinger_code/UDP_pinger_server.py`”是 ping 服务器程序的代码，无需你进行修改。在这个服务器代码中，一部分客户端数据包会被人为丢弃，用于模拟 UDP 通信中可能出现的无响应情况。

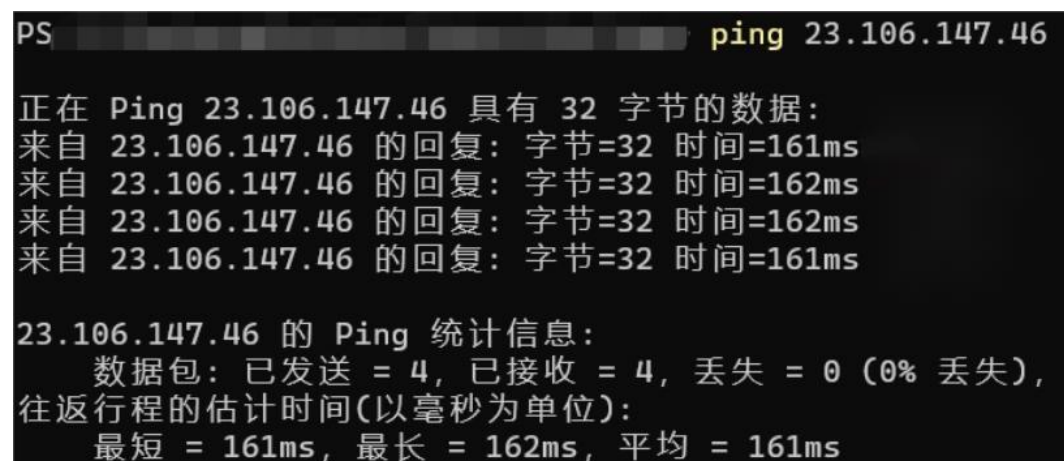
UDP 为应用程序提供了不可靠的传输服务，消息可能因为路由器队列溢出、硬件错误等其他原因，而在网络中丢失。但由于在内网环境中一般不容易观察到丢包，甚至数据包没有离开你本机（目的地址是本地的环回地址，`127.0.0.1`），所以本次实验的服务器程序通过随机不回复部分请求来模拟丢包。

ping 客户端代码要求：

客户端应向服务器发送 10 次基于 udp 的 ping。因为 UDP 是不可靠的协议，所以从客户端发送到服务器的数据包可能在网络中丢失。因此，客户端不能无限期地等待 ping 消息的回复。客户端一般会设置一个超时时间，如果在超时时间内没有收到回复，客户端程序会认为数据包在网络传输期间丢失。关于超时时间的设置，请自行查找 Python 文档，以了解如何在数据报套接字上设置超时值。

具体来说，你的 ping 客户端程序应该：

- 1) 使用 UDP 发送 ping 消息；
- 2) 超时时间设置成 5 秒；
- 3) 按照 windows 标准 ping 程序的格式进行输出，在控制台输出 ping 的结果，如下图所示（不需要实现 TTL）：



```
PS C:\> ping 23.106.147.46

正在 Ping 23.106.147.46 具有 32 字节的数据:
来自 23.106.147.46 的回复: 字节=32 时间=161ms
来自 23.106.147.46 的回复: 字节=32 时间=162ms
来自 23.106.147.46 的回复: 字节=32 时间=162ms
来自 23.106.147.46 的回复: 字节=32 时间=161ms

23.106.147.46 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 161ms, 最长 = 162ms, 平均 = 161ms
```

图 5：你的 ping 客户端程序输出示例

提交内容：

- 1、完整的客户端代码，保存为“udp_pinger_code/UDP_pinger_client.py”；
- 2、你所编写的 ping 客户端程序运行效果，将截图放到此处。



```
udp_pinger_code 29
py 30
UDP_pinger_server.py 31
UDP_pinger_client.py 32
UDP_pinger_server.py 33
web_server_code 34
UDP_pinger_client.py 35
正在 Ping 127.0.0.1 具有 32 字节的数据 36
请求超时。 37
Hello_SSE.html 38
simple_web_server.py 39
web_server_sample.py 40
-E208-实验3-实验要求-网络编程.docx 41
SSE208-实验3-实验要求-网络编程.docx 42
来自 127.0.0.1 的回复: 字节=32 时间<1ms 43
来自 127.0.0.1 的回复: 字节=32 时间<1ms 44
来自 127.0.0.1 的回复: 字节=32 时间<1ms 45
来自 127.0.0.1 的回复: 字节=32 时间<1ms 46
请求超时。 48
来自 127.0.0.1 的回复: 字节=32 时间<1ms 49

127.0.0.1 的 Ping 统计信息:
    数据包: 已发送 = 10, 已接收 = 7, 丢失 = 3 (30% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 0ms, 平均 = 0ms

~/obsidian_notes/25.2/cn/课/实验3-网络编程/udp_pinger_code main !185 ?55 >
```

思考题：请使用文字与截图（可选）回答

1) 结合本实验服务端会随机不回复的行为，说明 UDP ping 客户端为什么必须设置超时时间。如果不设置超时，程序可能会有什么问题？

UDP 是一种不可靠传输协议，它本身不保证数据一定送达，也不保证接收方一定会返回响应。在本实验的服务端程序中，服务器还会通过随机数人为模拟丢包行为：当随机数满足一定条件时，服务器收到客户端的数据报后不会发送任何回复。

2) 在本实验中，一次 ping 超时说明客户端在 5 秒内没有收到服务端回复。它是否一定说明真实网络中发生了丢包？还可能有哪些原因？

客户端发送的数据报可能在网络中丢失；服务端返回的响应数据报可能在返回途中丢失；网络拥塞或系统调度导致响应时间超过 5 秒；服务端程序没有启动、端口号设置错误、IP 地址设置错误；防火墙或系统安全策略拦截了 UDP 数据包；服务端接收缓冲区满或程序处理异常等。

四、参考阅读：

- 1、[硬核图解 TCP 的粘包问题](#)
- 2、[Python 网络编程](#)
- 3、[python socket 网络编程之粘包问题详解](#)

五、提交要求

1、实验报告：完成所有需要回答的部分（已使用**红字**指出），适当配合截图加以说明。你的回答应尽可能条理清晰，截图请缩放至适当大小；

2、提交截止时间：2026/6/3 中午 11:59:59

3、提交方式：将该实验报告**转换为 PDF，重命名为“学号-姓名-实验 3 实验报告”**，将三个代码文件夹（**“code”，“udp_pinger_code”，“web_server_code”**）和实验报告放在一起，**压缩为一个 zip 文件**，上传至坚果云，如下图所示。



4、提交链接：[实验课小实验报告统一收集](#)。注意**正确填写坚果云界面中要求的姓名、学号等字段**，系统会自动对 zip 压缩包进行重命名。